Research Project Report: Tasks 1 and 2

Plain-English Description, System Architecture, and Practical Results

Author: Olit Oljra **Date:** October 2026

Task 1: Verified Recursive Self-Improvement on Connect-4

1. What Was the Goal?

The goal of Task 1 was to answer a simple question: **Can an AI teach itself to get better at playing a game (Connect-4) purely through practice, without picking up bad habits or degrading?**

If an AI plays only against copies of itself without any guidance, it tends to fool itself. It makes a bad move, its opponent also makes a bad move, and the AI assumes its bad move was actually brilliant. Over time, it develops bad habits (called *policy collapse*) or forgets earlier lessons it had mastered (called *catastrophic forgetting*).

In this task, we built a system that allows the AI to practice freely, but pairs it with an infallible mathematical referee that checks every move and corrects its mistakes.

2. How the System Works (The 5 Main Building Blocks)

1. **The AI Player (Neural Network):** We created a convolutional neural network (CNN) that looks at the 7-column by 6-row Connect-4 board like an image. It detects horizontal lines, vertical stacks, and diagonal traps, and then scores which of the 7 columns is the best choice to drop a checker. If a column is already full to the top, the network automatically blocks it so it can never make an illegal move.
2. **The Infallible Referee (Pascal Pons C++ Solver):** Connect-4 is mathematically "solved," meaning math experts have calculated the proven best move for any board position. We connected our Python system to a high-speed C++ solver that evaluates boards in less than a millisecond. Every time the AI plays a practice move, this referee tells us the exact mathematical truth: was that move a winning move, a draw, or a blunder?
3. **Hunting for Mistakes (Mistake Mining):** Instead of wasting time practicing board positions the AI already understands well, we programmed the system to actively search for positions where the AI made a mistake. We dedicated up to 70% of each training round specifically to these weak spots, forcing the model to fix its errors.
4. **The Memory Bank (Experience Replay Buffer):** To ensure the AI didn't forget basic moves while learning new tricks, we kept a running memory bank. Each new training round mixed newly corrected mistakes with foundational boards learned earlier.
5. **The Safety Gate and Rollback:** After every practice round, the updated AI had to take a test on fresh, unseen boards. If its accuracy improved, the update was kept. If its score dropped, the system immediately rejected the update and rolled back to the previous good version. This guaranteed the AI never got worse.

3. What Were the Results?

We tested the AI across three independent runs (seeds 42, 43, 44) on Google Colab, comparing our solver-guided AI against a control AI that practiced without a referee. Both models saw the exact same number of training examples (15,000).

Table 1: Task 1 Results: Average performance across 3 runs on Colab.

Performance Metric	Without Referee	With Our Referee	Improv
Tournament Score (Live Matches)	49.2%	**61.1**%	+**11.9**% jun
Final Test Accuracy (Unseen Boards)	60.5%	**65.4**%	+**4.9**% a
Avoiding Blunders (Outcome Accuracy)	71.7%	**74.8**%	+**3.1**% fewer
Safety Gate Rollback Triggered?	Yes (constant drops)	No (steady growth)	Stable le

A Real Example of Improvement: On a critical test board, the initial untrained AI played Column 6—a fatal blunder that threw away an easy win and turned it into an immediate loss. After 3 practice rounds with our referee, faced with that exact same board, the AI found Column 3—the mathematically proven winning attack that forces a win in 17 moves.

—

Task 2: Low-Resource Amharic Capability Distillation

1. What Was the Goal?

The goal of Task 2 was to solve a major language limitation: **Small, lightweight AI models know almost nothing about low-resource languages like Amharic (Ethiopic Ge'ez script). Can we teach a tiny AI to understand and write Amharic by having it learn from a much larger, smarter model?**

Large AI models write good Amharic, but they require massive cloud computers and huge amounts of memory. Small models can run locally on laptops or phones, but out of the box, their Amharic text is completely broken and unusable.

2. How We Trained the Student Model

1. **The Teacher Model (`Gemma-3-4B-IT`):** A large Google model with 4.3 Billion parameters. We used this teacher to create 1,843 clean, high-quality question-and-answer pairs in Amharic.
2. **The Student Model (`Qwen2.5-0.5B-Instruct`):** A compact open-source model with only 494 Million parameters (nearly 10× smaller than the teacher).
3. **The Training Method (4-bit QLoRA):** Instead of retraining the whole model from scratch, we compressed the base model into 4-bit memory and attached a tiny trainable adapter.
    - We trained only **2.16 Million parameters** (just **0.43%** of the model).
    - The entire trained adapter takes up only **8.27 Megabytes** of disk space.
    - Training took only **17.7 minutes** on a single free Google Colab T4 GPU.

3. What the Model Looked Like Before vs. After

We evaluated both models on 60 brand-new, held-out Amharic prompts to see what the student actually learned:

- **Before Training (The Base Student):** The model was completely broken in Amharic:
    - It mixed random English and Portuguese letters inside Amharic words (e.g., writing gibberish like `anca` or `rão`).
    - It often repeated the user's question word-for-word instead of answering.
    - It could not form coherent grammatical sentences.
- **After Training (The Distilled Student):** The student showed a dramatic improvement:
    - **Clean Ethiopic Script:** It stopped mixing in Latin letters and wrote with over 98% pure Ethiopic script.
    - **Assistant Tone:** It learned how to respond like a polite assistant, using respectful greetings, bullet points, and concluding remarks.
    - **Grammar:** It adopted proper Amharic Subject-Object-Verb sentence structures.
    - **Lightweight Speed:** It generated answers in about 14 seconds using only **1.54 GB of GPU memory**, making it fast and cheap enough to run on edge devices.

4. Honest Limitations (What Still Needs Work)

While the student learned the alphabet, grammar, and tone of Amharic, two limitations remain:

1. **Factual Mistakes:** Because the model is small (494M parameters), it sometimes makes up incorrect facts while sounding confident.
2. **Repetitive Loops:** In about 20% of longer answers, the model gets stuck at the end, repeating the same concluding phrase multiple times.

These exact limitations provided the direct motivation for building an automated repetition checker in our follow-up research.